TECHNICAL REFERENCE

AI Global Architecture

Comprehensive System Design & Implementation Guide

Enhanced Edition with Detailed Explanations

Multi-Agent Systems | NLP | Computer Vision | Voice Processing

Design Patterns | Best Practices | Production Deployment

Version 2.0 | March 2026

Table of Contents

1. Agent Orchestration Architecture

- 1.1 Core Components Explained
- 1.2 State Machine Implementation
- 1.3 Best Practices

2. Agent Communication Protocols

- 2.1 Protocol Selection Guide
- 2.2 Message Flow Patterns

3. Individual Agent Specifications

- 3.1 Interview Agent Deep Dive
- 3.2 Vision Agent Implementation
- 3.3 Voice Agent Architecture
- 3.4 Scoring & Avatar Agents

4. NLP Architecture

- 4.1 Pipeline Components
- 4.2 Implementation Examples

5. Computer Vision Architecture

- 5.1 Real-time Processing Pipeline
- 5.2 Model Selection Criteria

6. Voice Processing Architecture

- 6.1 Audio Pipeline Design
- 6.2 Speech Analysis Techniques

7. TTS & Avatar Architecture

- 7.1 Neural Speech Synthesis
- 7.2 Lip Synchronization

8. RAG & Knowledge Architecture

- 8.1 Retrieval Pipeline
- 8.2 Knowledge Base Design

9. AI Design Patterns

- 9.1 Pattern Catalog

9.2 Memory Patterns

10. Model Serving Architecture

10.1 Infrastructure Design

10.2 Scaling Strategies

1. Agent Orchestration Architecture

> *The Agent Orchestration Architecture serves as the central nervous system of the AI interview platform, coordinating multiple specialized agents to deliver a seamless, coherent interview experience. This chapter provides comprehensive explanations of each component and practical implementation guidance.*

The orchestration layer addresses one of the most complex challenges in multi-agent AI systems: maintaining coherent state and context across distributed components while ensuring fault tolerance and scalability. The architecture employs the ReAct (Reasoning and Acting) pattern, which enables agents to reason about their actions, execute them, observe results, and adapt their strategy dynamically.

1.1 Core Components Explained

Agent Orchestrator

The Agent Orchestrator is the brain of the system, implementing a state machine engine based on LangGraph. This component provides deterministic workflow execution with support for branching logic, error recovery, and checkpoint-based state persistence.

Key Responsibilities:

- **Workflow Coordination:** Manages the sequence of interview phases (introduction, technical assessment, behavioral questions, conclusion)
- **Task Routing:** Directs specific tasks to appropriate agents based on their capabilities
- **State Transitions:** Handles transitions between interview states with validation and rollback capabilities
- **Error Recovery:** Implements fallback mechanisms when agents fail or timeout

Technology Stack: LangGraph / LangChain with custom state management extensions

Design Pattern: State Machine with hierarchical substates

Message Broker

The Message Broker serves as the central communication hub, implementing both publish-subscribe and request-reply patterns. This decouples agents from each other, allowing independent scaling and deployment.

Communication Patterns:

- **Pub/Sub:** For event broadcasting (e.g., interview phase changes, scoring updates)
- **Request-Reply:** For synchronous agent interactions (e.g., question generation requests)
- **Work Queues:** For load balancing across agent instances

Technology Options: Redis Streams (recommended for low latency), RabbitMQ (for complex routing), Apache Kafka (for high throughput)

State Manager

The State Manager maintains conversation history, candidate context, and interview progress. It uses a hybrid storage approach optimized for different access patterns.

Storage Strategy:

- **Redis (Hot Storage):** Fast access to active session data, conversation buffers, and real-time context
- **PostgreSQL (Warm Storage):** Persistent storage for completed interviews, historical data, and analytics
- **Object Storage (Cold):** Audio/video recordings, large model outputs

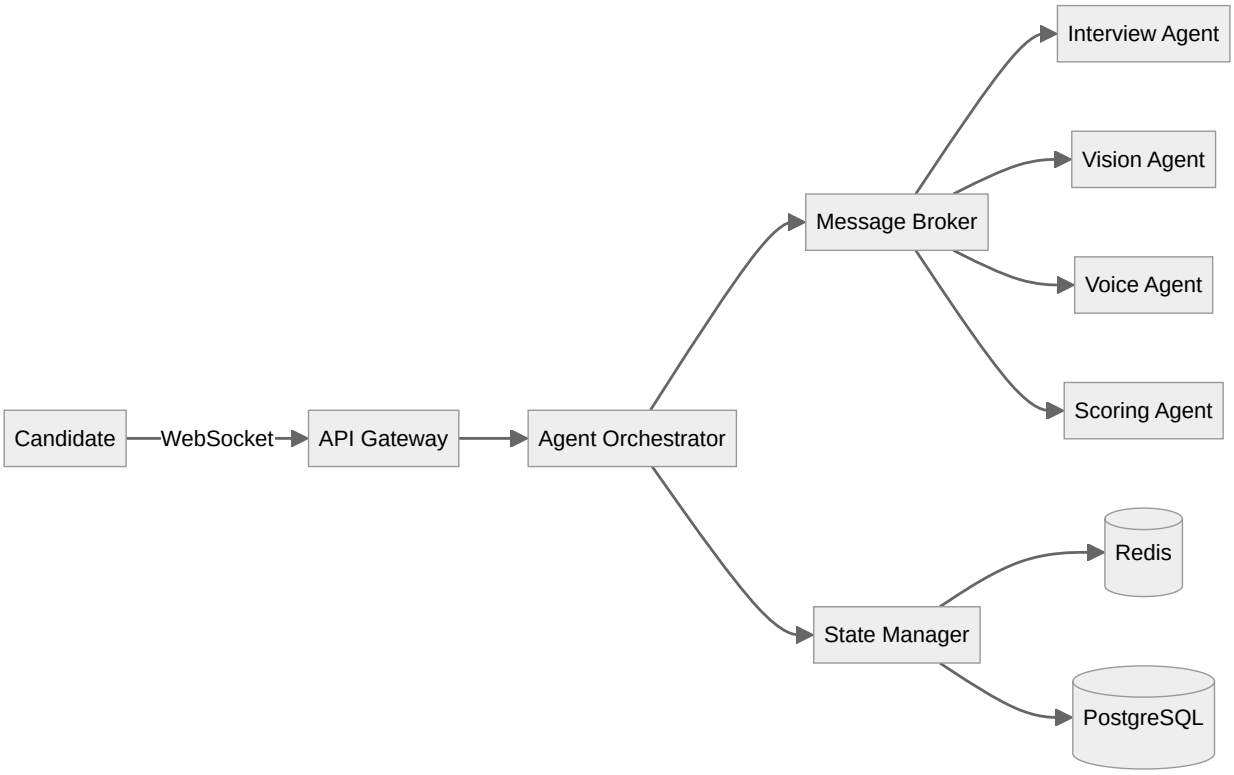

Figure 1: Agent Orchestration High-Level Architecture

1.2 State Machine Implementation

The state machine implementation uses LangGraph to define interview flow as a directed graph where nodes represent states and edges represent transitions. This approach provides several advantages:

> **Why State Machines for AI Orchestration?**
>
> State machines provide **deterministic execution** in an otherwise non-deterministic AI environment. While LLM outputs are probabilistic, the orchestration layer must be predictable and auditable. State machines also enable **easy visualization** of interview flows, **simplified debugging** through state inspection, and **automatic checkpointing** for fault tolerance.

Core States in Interview Flow:

Table 1: Interview State Definitions

State	Description	Transitions
INIT	Session initialization, candidate verification	→ WELCOME
WELCOME	Introduction and context setting	→ TECHNICAL or → BEHAVIORAL
TECHNICAL	Technical skills assessment	→ TECHNICAL (loop) or → BEHAVIORAL
BEHAVIORAL	Soft skills and culture fit evaluation	→ BEHAVIORAL (loop) or → CLOSING
CLOSING	Final remarks and next steps	→ COMPLETED
COMPLETED	Interview finalized, scoring initiated	Terminal state

1.3 Best Practices

Checkpoint Strategy

Implement checkpoints at every state transition to enable recovery from failures. Store the complete state (conversation history, candidate context, agent outputs) at each checkpoint. This allows resuming interviews from the exact point of failure without data loss.

Timeout Handling

Define explicit timeouts for each state:

- Agent response timeout: 30 seconds
- Candidate response timeout: 5 minutes (configurable)
- Overall interview timeout: 60 minutes

Implement graceful degradation - if an agent times out, fall back to a simpler response or skip the question rather than failing the entire interview.

Observability

Instrument the orchestrator with comprehensive logging and metrics:

- State transition logs with timestamps
- Agent response times and success rates
- Message queue depths and processing latencies
- Error rates by agent and state

2. Agent Communication Protocols

> *Communication protocols define how agents exchange information, coordinate tasks, and maintain consistency. The choice of protocol significantly impacts system performance, reliability, and scalability.*

2.1 Protocol Selection Guide

Selecting the appropriate protocol for each interaction type is crucial for system performance. The following guide helps make informed decisions:

Table 2: Protocol Selection Matrix

Scenario	Recommended Protocol	Rationale
Real-time chat with candidate	WebSocket	Low latency, bidirectional, persistent connection
Agent-to-agent RPC	gRPC	Binary serialization, HTTP/2 multiplexing, strong typing
External API integration	HTTP/REST	Universal compatibility, caching support
Event broadcasting	Redis Streams	Pub/sub simplicity, persistence, consumer groups
Reliable message delivery	AMQP	Guaranteed delivery, dead-letter queues, routing

Protocol Deep Dive: gRPC for Inter-Agent Communication

gRPC is the recommended protocol for internal agent communication due to its performance characteristics:

> **gRPC Advantages**
>
> - **Protocol Buffers:** Binary serialization is 3-5x faster than JSON with smaller payload sizes
> - **HTTP/2:** Multiplexing allows multiple concurrent requests over single connection
> - **Streaming:** Supports server, client, and bidirectional streaming for real-time data
> - **Code Generation:** Automatic client/server stub generation from proto definitions

Example Proto Definition:

```
syntax = "proto3";
package interview;

service InterviewAgent {
  rpc GenerateQuestion(QuestionRequest) returns (QuestionResponse);
  rpc AnalyzeResponse(ResponseRequest) returns (AnalysisResponse);
  rpc StreamConversation(stream ChatMessage) returns (stream ChatMessage);
}

message QuestionRequest {
  string domain = 1;
  int32 difficulty = 2;
  string context = 3;
}

message QuestionResponse {
  string question = 1;
  repeated string follow_up_hints = 2;
  int32 estimated_time_seconds = 3;
}
```

2.2 Message Flow Patterns

Understanding message flow patterns is essential for designing robust agent interactions. The following patterns address common coordination scenarios:

Request-Response Pattern

The simplest pattern where Agent A sends a request to Agent B and waits for a response. Used for synchronous operations like question generation or response analysis.

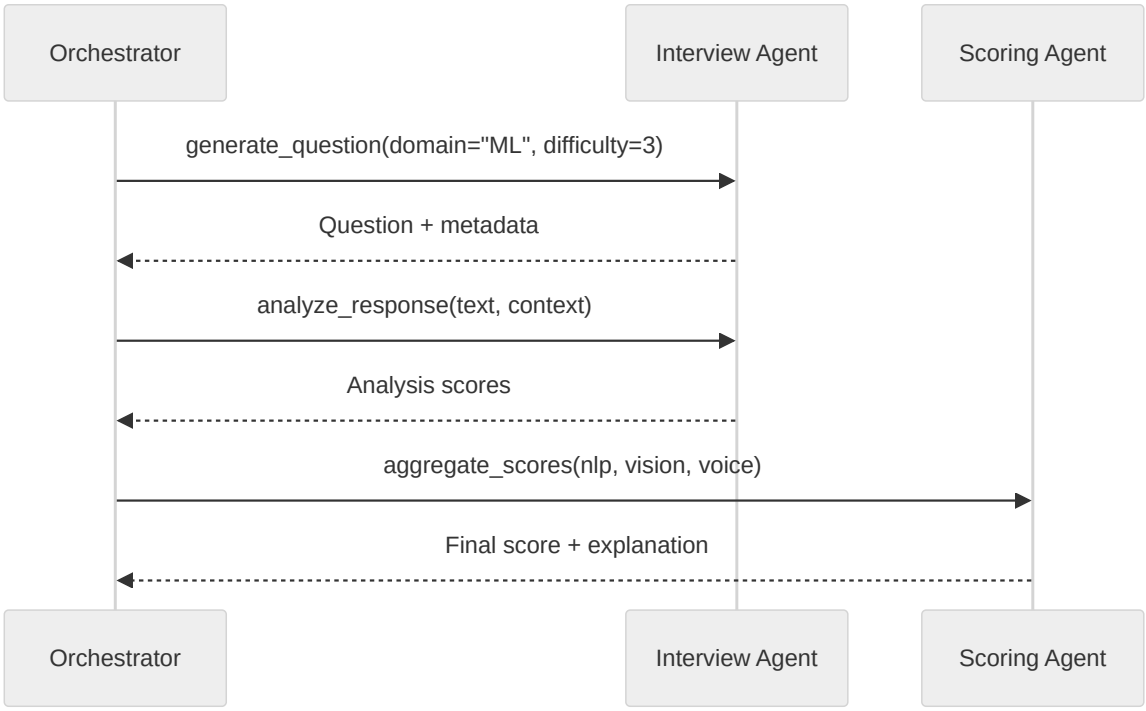

Figure 2: Request-Response Message Flow

Event-Driven Pattern

Agents publish events to a message broker without waiting for responses. Other agents subscribe to relevant events and react asynchronously. This pattern enables loose coupling and better scalability.

> **Event Sourcing Benefits**
>
> Event sourcing stores all state changes as a sequence of events rather than just the current state. This provides:
>
> - Complete audit trail of all system activities
> - Ability to replay events for debugging or recovery
> - Temporal queries ("What was the state at time T?")
> - Easy integration with stream processing for analytics

Table 3: Message Type Reference

Type	Direction	Purpose	Example
REQUEST	Agent → Agent	Action invocation	analyze_response(text, context)
RESPONSE	Agent → Agent	Result return	score: 0.85, sentiment: positive
EVENT	Broadcast	State change notification	interview_phase_changed
COMMAND	Orchestrator → Agent	Direct instruction	start_evaluation(candidate_id)
QUERY	Agent → State Manager	Information retrieval	get_interview_context(session_id)
ACK	Receiver → Sender	Delivery confirmation	message_id: 12345, status: received

3. Individual Agent Specifications

> *Each agent in the system is designed as an autonomous, specialized component with clearly defined responsibilities. This section provides detailed specifications and implementation guidance for each agent type.*

3.1 Interview Agent Deep Dive

The Interview Agent is the primary conversational AI responsible for conducting the interview dialogue. It leverages Large Language Models fine-tuned for interview contexts to generate contextually relevant questions and analyze responses.

Core Components

LLM Chat Engine

The foundation of the Interview Agent, providing natural language understanding and generation capabilities. Supports multiple backends (GPT-4 Turbo, Claude 3 Opus, Llama 3) for flexibility and redundancy.

Question Generator

Specialized module for creating interview questions based on domain, difficulty level, and candidate profile. Uses few-shot prompting with examples of high-quality questions.

Context Manager

Maintains conversation history and relevant context throughout the interview. Implements sliding window and summarization strategies to fit within LLM context limits.

RAG Retrieval

Retrieves relevant information from knowledge bases to ground questions and evaluations in company-specific requirements and best practices.

Response Analyzer

Evaluates candidate responses for technical accuracy, completeness, clarity, and alignment with job requirements.

Table 4: Interview Agent Specification

Category	Details
Tools	generate_question(), analyze_response(), adapt_flow(), score_answer(), generate_followup()
Inputs	Candidate Profile, Job Requirements, Domain Context, Interview State, Previous Responses
Outputs	Questions, Follow-up Queries, Response Scores, Phase Transitions
Protocols	WebSocket (real-time), gRPC (internal), HTTP/REST (config)
LLM Backends	GPT-4 Turbo (primary), Claude 3 Opus (fallback), Llama 3 (on-premise)

Context Window Management

LLMs have limited context windows (4K-128K tokens depending on model). For long interviews, implement:

- **Sliding Window:** Keep only the most recent N turns
- **Summarization:** Compress older conversation into summaries
- **Key Extraction:** Extract and store critical facts separately

3.2 Vision Agent Implementation

The Vision Agent processes video streams to extract behavioral signals and emotional indicators. It employs state-of-the-art computer vision models for real-time analysis.

Processing Pipeline

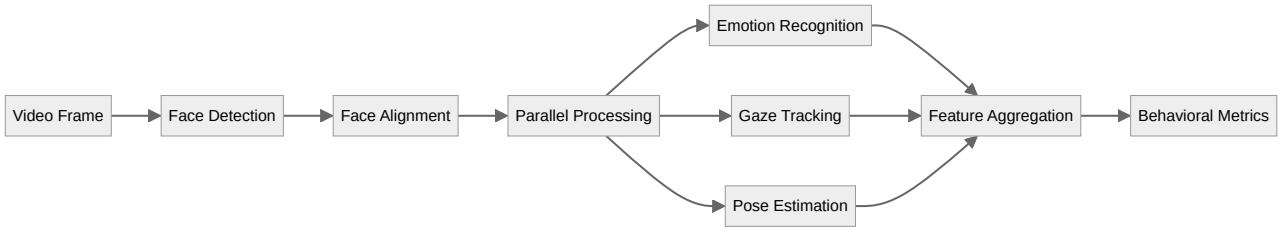

Figure 3: Vision Agent Processing Pipeline

Model Selection Rationale

Table 5: Vision Models Comparison

Task	Model	Latency	Accuracy	Notes
Face Detection	RetinaFace	15ms	99.2% AP	Handles occlusion, multiple faces
Emotion Recognition	EfficientNet-B2	8ms	76% FER+	7 basic emotions + valence/arousal
Gaze Tracking	MediaPipe Face Mesh	5ms	3-5 deg error	468 facial landmarks
Pose Estimation	BlazePose	12ms	33 keypoints	Real-time on CPU

Attention Score Calculation

The attention score (0-100) is computed from multiple signals:

```
attention_score = w1 * gaze_centered + w2 * blink_rate_normal +
                  w3 * head_stability + w4 * facial_engagement

Where:
- gaze_centered: Percentage of time looking at camera (0-1)
- blink_rate_normal: Deviation from normal blink rate (12-20/min)
- head_stability: Low head movement indicates focus
- facial_engagement: Presence of responsive expressions
```

3.3 Voice Agent Architecture

The Voice Agent handles all audio processing including automatic speech recognition, speaker diarization, prosodic analysis, and stress detection.

Audio Pipeline Components

Voice Activity Detection (VAD)

Detects presence of human speech using Silero VAD or WebRTC VAD. Essential for segmenting audio and reducing processing load by only analyzing speech segments.

Automatic Speech Recognition (ASR)

Converts speech to text using Whisper Large-v3. Supports 99 languages with word-level timestamps. Handles accents, background noise, and domain-specific terminology through fine-tuning.

Speaker Diarization

Segments audio by speaker identity using Pyannote Audio. Distinguishes between interviewer and candidate in two-way conversations.

Prosody Analysis

Extracts pitch contour, speech rate, pause patterns, and intensity variations. These features indicate confidence, enthusiasm, and stress levels.

Speech Emotion Recognition (SER)

Detects emotional states from acoustic properties using Wav2Vec2-based models fine-tuned on emotion datasets.

Real-time Processing Optimization

For live interviews, process audio in chunks (e.g., 500ms) rather than waiting for complete utterances:

- Use streaming ASR for immediate transcription
- Apply VAD to detect speech boundaries
- Buffer chunks and process in parallel
- Return partial results for responsiveness

3.4 Scoring and Avatar Agents

Scoring Agent

The Scoring Agent aggregates signals from all other agents into a comprehensive candidate assessment. It implements weighted scoring algorithms, ML ranking models, and explainability mechanisms.

Table 6: Scoring Agent Components

Component	Technology	Purpose
ML Ranker	XGBoost	Learns optimal feature weights from historical hiring decisions
Explainability Engine	SHAP	Provides feature importance for each scoring decision
Calibration Model	Platt Scaling	Converts model outputs to probability scores
Report Generator	Jinja2 templates	Creates structured evaluation reports

Avatar Agent

The Avatar Agent generates a realistic AI interviewer with synchronized speech, facial expressions, and lip movements.

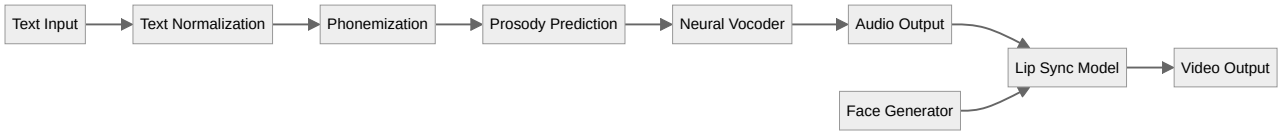

Figure 4: Avatar Generation Pipeline

4. NLP Architecture

> *The Natural Language Processing architecture encompasses all text understanding and generation capabilities. This section details the multi-stage pipeline and implementation considerations.*

4.1 Pipeline Components

The NLP pipeline processes text through multiple stages, each adding semantic understanding:

1. Tokenization

Breaking text into tokens (words, subwords, or characters). Modern transformer models use WordPiece or Byte Pair Encoding (BPE) algorithms. Tokenization also includes sentence boundary detection and language identification.

2. Named Entity Recognition (NER)

Identifies and classifies named entities such as skills, organizations, locations, dates, and custom entities relevant to recruitment. Uses transformer-based models fine-tuned on domain-specific data.

> **Custom Entity Types for Recruitment**
>
> - **SKILL:** Programming languages, frameworks, tools (Python, TensorFlow, Kubernetes)
> - **EXPERIENCE:** Years of experience, seniority levels
> - **EDUCATION:** Degrees, institutions, certifications
> - **ROLE:** Job titles and positions
> - **PROJECT:** Project names and descriptions

3. Sentiment Analysis

Determines the emotional tone and opinion expressed in candidate responses. Implements aspect-based sentiment analysis to evaluate specific aspects of answers.

4. Semantic Similarity

Computes similarity between text segments using dense vector representations. Enables comparison of candidate responses against reference answers and semantic search across the knowledge base.

5. Text Generation

Produces natural language text using Large Language Models for question generation, summarization, and paraphrasing.

4.2 Implementation Examples

Response Scoring with Semantic Similarity:

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('all-MiniLM-L6-v2')

def score_response(candidate_answer, reference_answer):
    # Encode both answers
    candidate_emb = model.encode(candidate_answer)
    reference_emb = model.encode(reference_answer)

    # Compute cosine similarity
    similarity = cosine_similarity(candidate_emb, reference_emb)

    # Scale to 0-100 score
    return similarity * 100
```

5. Computer Vision Architecture

> *The Computer Vision architecture processes video streams to extract behavioral and emotional signals. The pipeline is optimized for low-latency real-time processing.*

5.1 Real-time Processing Pipeline

The vision pipeline must process video at 30 FPS with end-to-end latency under 100ms for real-time feedback. This requires careful optimization:

Performance Optimization Strategies

- **Model Quantization:** Convert FP32 models to INT8 for 2-4x speedup with minimal accuracy loss
- **Batch Processing:** Process multiple frames together when possible
- **GPU Acceleration:** Use TensorRT or ONNX Runtime for optimized inference
- **Region of Interest:** Only process face regions, not full frames
- **Model Cascades:** Use lightweight models for initial detection, heavy models for refinement

5.2 Model Selection Criteria

Table 7: Vision Model Selection Guide

Criteria	Importance	Evaluation Method
Inference Speed	Critical	Measure FPS on target hardware
Accuracy	High	Evaluate on benchmark datasets
Model Size	Medium	Consider deployment constraints
Robustness	High	Test with varying lighting, angles
License	Medium	Ensure commercial use is permitted

6. Voice Processing Architecture

> *Voice processing transforms raw audio into structured, analyzable data enriched with prosodic and emotional information.*

6.1 Audio Pipeline Design

The audio pipeline handles the complete flow from raw audio input to structured analysis output:

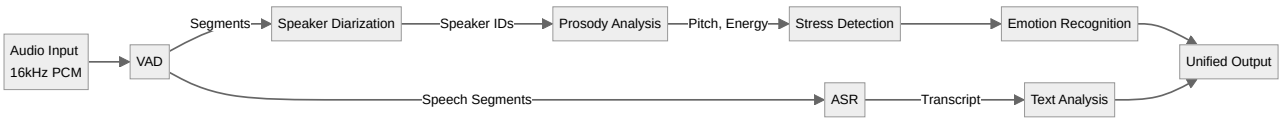

Figure 5: Voice Processing Pipeline

6.2 Speech Analysis Techniques

Stress Detection

Stress is detected through multiple acoustic features:

- **Speaking Rate:** Faster speech often indicates stress
- **Pitch Variability:** Increased pitch variation under stress
- **Pause Patterns:** Unnatural pauses or lack of pauses
- **Voice Quality:** Jitter and shimmer measurements
- **Hesitation Markers:** Fillers ("um", "uh"), repetitions

> **Confidence Scoring from Speech**
>
> Confidence can be inferred from:
>
> - Low filler word count
> - Consistent speaking pace
> - Clear articulation (high spectral clarity)
> - Appropriate pause placement
> - Steady pitch without excessive variation

7. TTS & Avatar Architecture

> *The Text-to-Speech and Avatar architecture creates realistic AI interviewers with synchronized facial animation for engaging, human-like interactions.*

7.1 Neural Speech Synthesis

Modern TTS systems use neural approaches for natural-sounding speech:

TTS Pipeline Stages

Text Normalization

Preprocesses input text for consistent pronunciation: expands abbreviations, converts numbers to words, handles dates and currencies.

Phonemization

Converts graphemes to phonemes using eSpeak NG or neural G2P models for accurate pronunciation.

Prosody Prediction

Predicts duration, pitch contour, and energy for each phoneme using neural networks trained on annotated speech.

Neural Vocoder

Generates waveform from mel-spectrograms using HiFi-GAN, BigVGAN, or similar architectures.

7.2 Lip Synchronization

Lip sync models animate face images to match audio speech:

Table 8: Lip Sync Model Comparison

Model	Quality	Speed	Best For
Wav2Lip	Good	Real-time	Live applications
SadTalker	Excellent	Near real-time	High-quality output
First Order Motion	Very Good	Real-time	Expression control

8. RAG & Knowledge Architecture

> *Retrieval-Augmented Generation provides the knowledge foundation for intelligent question generation and response evaluation by combining dense vector retrieval with generative models.*

8.1 Retrieval Pipeline

The RAG pipeline transforms user queries into contextually relevant, factually grounded content:

Figure 6: RAG Pipeline Flow

Query Processing Techniques

- **Query Rewriting:** Expand or clarify ambiguous queries
- **Query Expansion:** Add relevant terms using synonym expansion
- **Hypothetical Document Embedding (HyDE):** Generate hypothetical answer and use its embedding for retrieval
- **Query Routing:** Direct searches to appropriate knowledge bases

8.2 Knowledge Base Design

Table 9: Knowledge Source Definitions

Source	Content Type	Update Frequency	Use Case
Job Descriptions	Role requirements, skills, responsibilities	On-demand	Question generation, evaluation criteria
Candidate Profiles	Resumes, experience, education	Per interview	Personalized questions, profile matching
Interview History	Past questions, responses, scores	Real-time	Adaptive questioning, consistency checks
Domain Knowledge	Technical concepts, best practices	Monthly	Technical question validation, answer scoring
HR Guidelines	Company policies, evaluation rubrics	Quarterly	Bias prevention, standardized criteria

Chunking Strategy

Effective document chunking is critical for retrieval quality:

- **Size:** 256-512 tokens per chunk (balance between specificity and context)
- **Overlap:** 10-20% overlap between chunks to preserve context
- **Boundaries:** Chunk at semantic boundaries (paragraphs, sections) when possible
- **Metadata:** Include source, date, and category with each chunk

9. AI Design Patterns

> *Established design patterns ensure maintainability, scalability, and reliability in AI system development. These patterns represent proven solutions adapted for the recruitment interview context.*

9.1 Pattern Catalog

ReAct Pattern (Reasoning + Acting)

Combines reasoning and acting in an interleaved manner. The agent reasons about what action to take, executes the action, observes the result, and reasons again.

> **ReAct Example Flow**
>
> 1. **Reason:** "I need to assess the candidate's Python skills"
> 2. **Act:** Generate Python question about data structures
> 3. **Observe:** Candidate provided an answer about lists vs tuples
> 4. **Reason:** "The answer shows basic understanding but lacks depth on mutability"
> 5. **Act:** Generate follow-up about memory implications

Plan-and-Execute

Separates planning from execution. First generates a complete plan of actions, then executes them sequentially. Useful for complex, multi-step tasks where upfront planning improves efficiency.

Multi-Agent Collaboration

Distributes tasks among specialized agents that work together. Each agent has specific expertise (NLP, Vision, Voice) and they communicate to solve complex problems collaboratively.

RAG Pattern

Augments LLM generation with retrieved context from a knowledge base. Reduces hallucinations and enables knowledge updates without model retraining.

Chain of Thought

Elicits reasoning by prompting the model to show its work. Improves performance on complex reasoning tasks by making the inference process explicit.

9.2 Memory Patterns

Table 10: Memory Pattern Comparison

Pattern	Description	Best For	Limitations
Conversation Buffer	Sliding window of recent messages	Short conversations	Linear growth, loses old context
Summary Memory	Compresses history into summaries	Long conversations	May lose important details
Entity Memory	Extracts and tracks entities	Fact-heavy conversations	Requires NER, entity schema
Vector Store Memory	Stores turns as embeddings	Very long histories	Retrieval may miss relevant context

10. Model Serving Architecture

> *The Model Serving architecture provides infrastructure for deploying, managing, and scaling ML models in production with optimized inference and resource utilization.*

10.1 Infrastructure Design

Inference Server

Triton Inference Server or TorchServe for model deployment:

- Supports multiple frameworks (PyTorch, TensorFlow, ONNX)
- Dynamic batching for throughput optimization
- Model ensemble support for complex pipelines
- Built-in metrics and health checks

GPU Layer

NVIDIA A100/H100 GPUs with optimization:

- CUDA and TensorRT for kernel optimization
- Multi-instance GPU (MIG) for resource partitioning
- Model parallelism for large models

Model Registry

MLflow for model versioning and lifecycle management:

- Version tracking with artifact storage
- Model promotion workflows (dev → staging → prod)
- A/B testing support
- Rollback capabilities

10.2 Scaling Strategies

Table 11: Scaling Approaches

Strategy	When to Use	Implementation
Horizontal Scaling	High request volume	Kubernetes HPA based on GPU utilization
Vertical Scaling	Large model inference	Upgrade to larger GPU instances
Model Quantization	Latency constraints	INT8/FP16 conversion with TensorRT
Caching	Repeated queries	Redis for embedding cache
Batching	Throughput optimization	Dynamic batching in Triton

Production Checklist

- Model performance monitoring (latency, throughput, error rates)
- Drift detection for input and prediction distributions
- A/B testing framework for model updates
- Automated rollback on error threshold breach
- Cost monitoring and optimization alerts

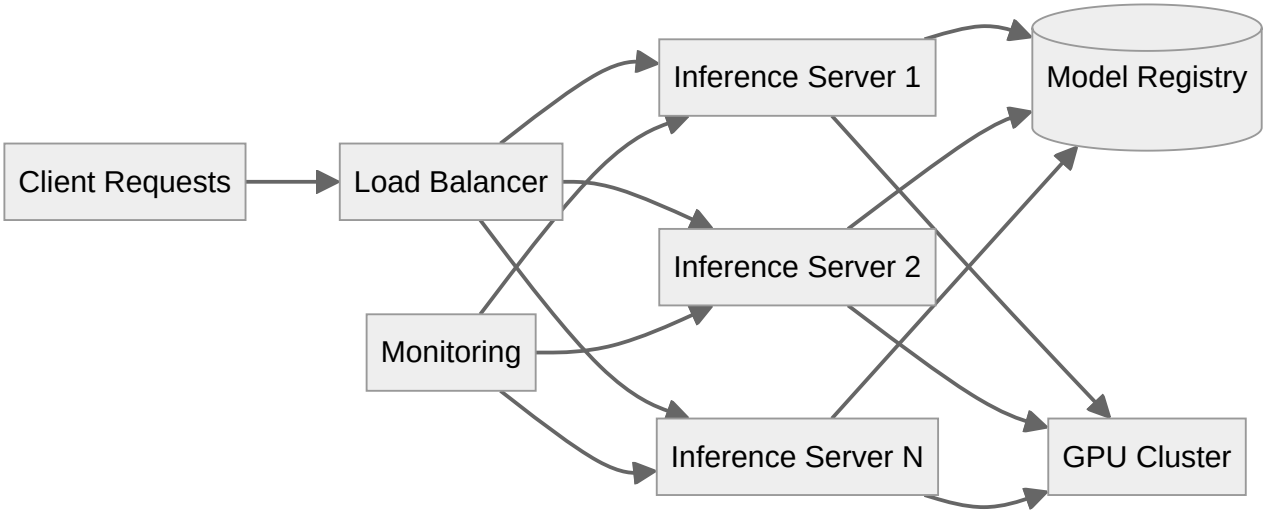

Figure 7: Model Serving Architecture